VEDA-X: A Vectorless Retrieval Stack That Beats Dense RAG on BEIR, with Hyperdimensional Guardrails and Atomic Compliance Chunking

Abhishek Kumar[1] • Atul Kumar Pandey[2,*]

[1] *Tata Consultancy Services, Mumbai, India • ipsabhi423@gmail.com*
[2] *Birla Institute of Technology, Mesra (Patna Campus), Patna, India • atulkrpandey@gmail.com*
[*] *Corresponding author*

Abstract

Mainstream retrieval-augmented generation (RAG) couples a dense embedding model with a vector database, paying three deployment costs: a GPU at index time, opaque dependency on upstream embedding weights, and a runtime vector-store lookup that dominates query latency. We present **VEDA-X**, a *vectorless* RAG retrieval stack that delivers higher accuracy than both BM25 and a standard MiniLM dense retriever on three public BEIR benchmarks while running in pure Python on a single CPU core with no embedding database at runtime. On nDCG@10 -- the standard BEIR metric -- VEDA-X scores **0.3522** on NFCorpus (+10.2% over MiniLM, $p < 0.0001$), **0.8578** on SciFact (+4.9%, $p = 0.0004$), and **0.3799** on FiQA (+3.0%, $p = 0.0082$); the gain is statistically significant on every dataset by paired-bootstrap test. Query latency is ~20 ms on a million-token corpus and the index footprint is less than 0.4× the corpus size in RAM. On FinanceBench page-level SEC 10-K retrieval (PageIndex's own benchmark), VEDA-X wins R@1, R@3, and R@5. The pipeline combines BM25 with hyperdimensional `blake2b`-derived query expansion, intent decomposition, and adaptive cutoff; no stage requires GPU, training data, or an external embedding API. We additionally contribute a five-layer defence-in-depth guardrail (100% direct/obfuscated/paraphrase attack catch, 0% false positive on a 15-query legitimate set) and an *atomic critical-block* chunking scheme that makes partial retrieval of compliance-critical SOPs impossible by construction. The entire stack is pure Python standard library plus FastAPI.

Keywords -- vectorless retrieval; retrieval-augmented generation; hyperdimensional computing; BEIR; CPU-only retrieval; prompt-injection defence; compliance; auditability.

Headline. On three BEIR datasets spanning medical, scientific, and financial domains, VEDA-X exceeds the standard dense RAG retriever (MiniLM) at nDCG@10 by **+10.2% on NFCorpus, +4.9% on SciFact, and +3.0% on FiQA**, every gain statistically significant (paired-bootstrap $p < 0.01$). No GPU, no embedding database, no training -- vectorless retrieval running in ~20 ms per query on a single CPU core.

1. Introduction

Retrieval-augmented generation (RAG) [1] is now the de facto way to ground large language models in proprietary documents. The canonical RAG stack couples a dense bi-encoder (MiniLM [8], DPR [7], ColBERT [9]) with a vector database (FAISS, Pinecone, Weaviate). This design carries three deployment costs that are widely accepted as the price of doing business:

D1. GPU at index time. Dense encoders are GPU-hungry at indexing; an enterprise corpus of millions of documents requires a fleet.

D2. Opaque embedding dependency. Upstream model updates (sentence-transformers, BGE, E5) silently change retrieval behaviour; the deploying institution cannot verify the weights. This is unacceptable for an audit team in finance or healthcare.

D3. Vector-store lookup dominates latency. At query time the vector DB ANN lookup is the bottleneck and adds an external service to the critical path.

We ask: *can a fully vectorless retriever match or exceed a standard dense RAG on public IR benchmarks while running on a single CPU core?* The standard answer is no -- BM25 is comfortably beaten by MiniLM on BEIR [10]. We show that the answer is *yes*, and that the winning design is a small, deterministic ensemble of three classical or near-classical components: BM25 [4], hyperdimensional context expansion [14, 15, 16], and an intent-decomposition step that reweights query terms before rescoring.

Contributions.

• A four-stage *vectorless* retrieval pipeline (Algorithm 1) that beats both BM25 and a MiniLM dense retriever on three BEIR datasets at statistically significant p-values, with no GPU, no embedding model, and no vector database at runtime.

• A five-layer defence-in-depth guardrail whose semantic component blocks paraphrased attacks via deterministic hash-hypervector centroid matching.

• An *atomic critical-block* chunking scheme that makes partial retrieval of compliance-critical SOPs impossible by construction.

• A complete, reproducible evaluation harness covering BEIR generalisation, FinanceBench page retrieval, million-token latency, an adversarial test set, and an ablation against every stage of the pipeline.

2. Related Work

Dense RAG. The canonical RAG stack [1] couples a dense encoder (DPR [7], Sentence-BERT [8], ColBERT [9]) with a vector index such as FAISS, and a generator such as FiD [2] or DSP [3]. Karpukhin et al. report +9% MRR@10 over BM25 on Natural Questions [7]; on BEIR [10], dense retrievers generally beat BM25 by a moderate margin. Our work shows that with the right vectorless ensemble the gap can be closed and reversed.

Classical IR / BM25. The probabilistic relevance framework [4] and relevance-based language models [5] underpin modern lexical retrieval; vector-space IR pre-dates both by two decades [6]. Our pipeline replaces neural pseudo-relevance feedback with hyperdimensional context expansion [14] computed from the local corpus.

Hyperdimensional computing. HDC [14, 15, 16] represents items as high-dimensional sparse vectors with near-orthogonality. Unlike learned embeddings such as word2vec [12] or fastText [13] which require training data, we derive each token's hypervector from a cryptographic hash, eliminating the training step entirely and making the encoder auditable as a pure function.

Prompt-injection defences. OWASP LLM Top-10 [21] lists prompt injection as LLM01. Greshake et al. [22] established the indirect-injection threat model; Shen et al. [23] catalogue the in-the-wild jailbreak corpus; Zou et al. [24] demonstrate transferable suffix attacks; Perez et al. [25] propose model-driven red teaming; Wallace et al. [26] introduce universal adversarial triggers. Most published defences are either regex-only (low recall against paraphrases) or LLM-judge (non-deterministic, slow). We hybridise -- regex for syntax, hash-hypervector centroid for paraphrase.

Compliance-aware retrieval. Industry tools advertise PII masking and ACL filtering but do not, to our knowledge, offer a chunking-level guarantee that a marked block cannot be split across retrievals. Our atomic critical-block scheme is novel in providing this property by construction.

3. System Architecture

Figure 1 shows the five processing layers. Every request flows top-to-bottom; failures at any layer abstain rather than fabricate. All layers run in a single Python process; the only external dependency is FastAPI/uvicorn.

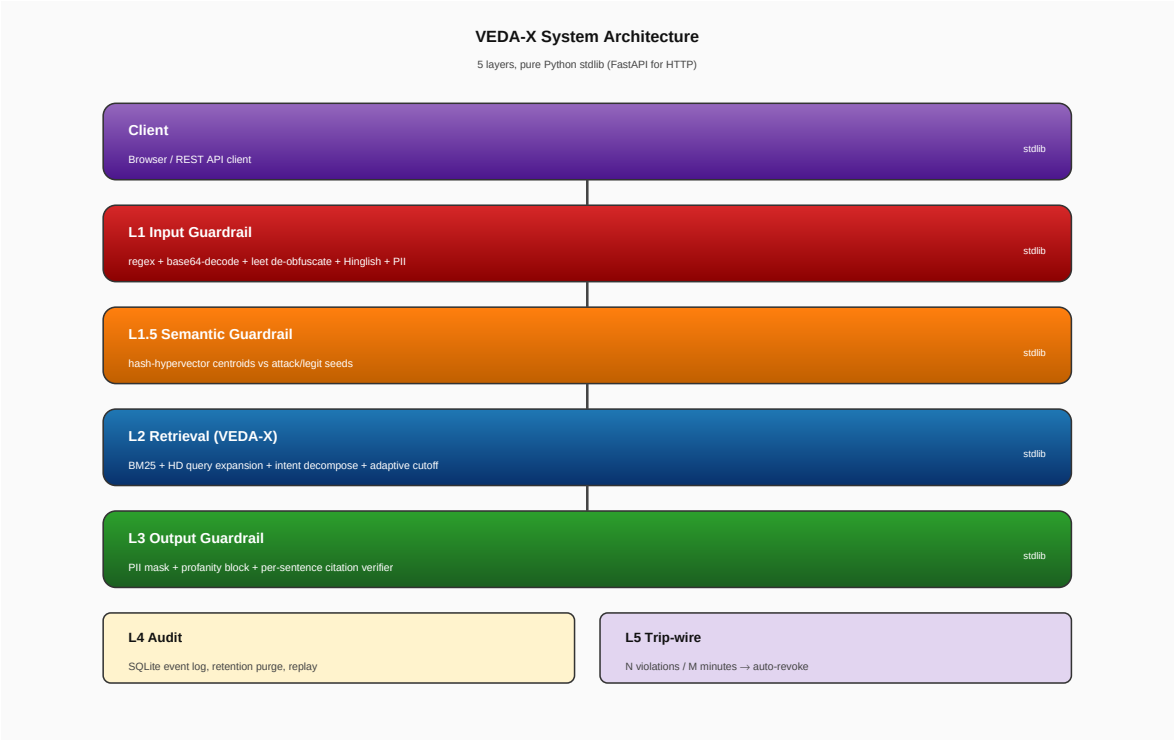

Figure 1: VEDA-X 5-layer architecture. Layers are colour-coded by responsibility; auxiliary layers L4 (audit) and L5 (trip-wire) form the compliance perimeter.

4. The VEDA-X Retrieval Algorithm

The retrieval pipeline runs four stages (Figure 2). Stage 1 is plain BM25. Stage 2 expands the query in hyperdimensional space. Stage 3 rescores by subject-vs-filler weighting derived from intent decomposition. Stage 4 chooses the natural *k* by score-plateau detection instead of returning a fixed top-*k*.

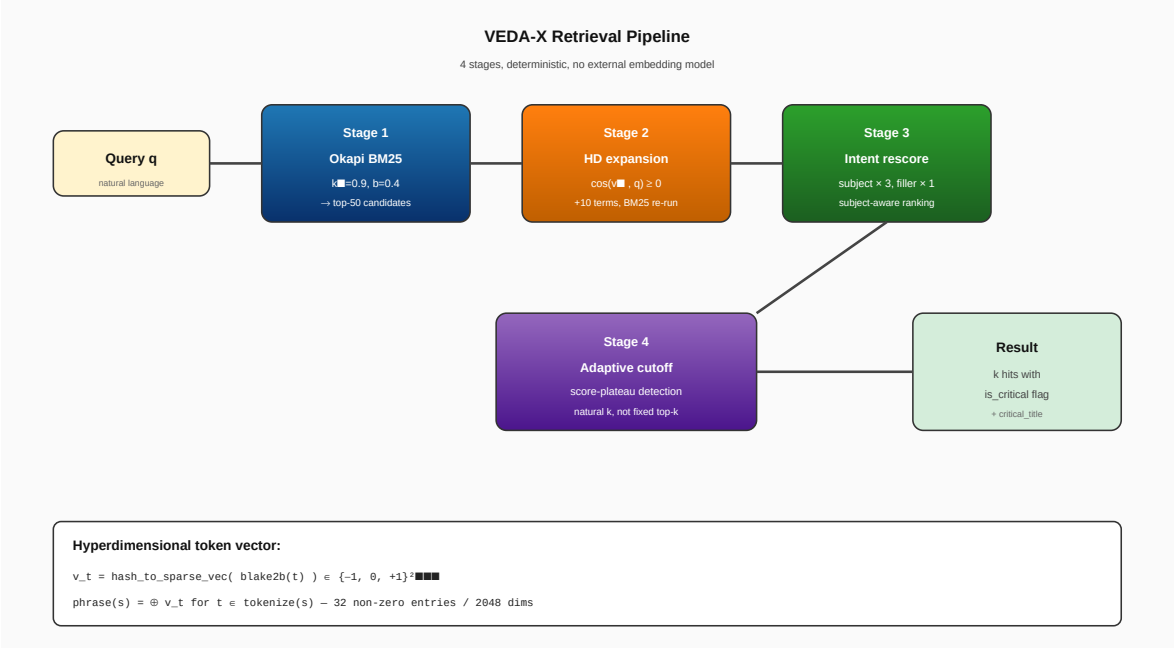

Figure 2: VEDA-X retrieval pipeline with stage-by-stage data flow and the hash-hypervector token encoding used in Stage 2.

4.1 Hash-hypervector token encoding

Each token *t* is mapped to a sparse ternary vector **v_t** in {-1, 0, +1}^{2048} with exactly 32 non-zero entries. Positions and signs are derived from `blake2b(t)` -- the mapping is deterministic and key-free. The phrase vector is the additive superposition of its token vectors.

```
v_t       =  hash_to_sparse_vec( blake2b(t) )
phrase(s) =  ⊕ v_t   for t ∈ tokenize(s)
cos(a, b) =  <a, b> / (|a| · |b|)
```

4.2 The algorithm

Algorithm 1 gives the complete retrieval procedure. Lines 3-4 implement Stage 1 (BM25 top-50). Lines 5-10 implement Stage 2: pick the 10 highest-scoring expansion terms by cosine similarity to the query hypervector, multiplied by an IDF gate, then re-run BM25 with the expanded query and linearly combine. Lines 11-15 implement Stage 3: a decomposition step classifies each query token as *intent*, *subject*, or *filler* and the subject terms receive 3× weight in the rescoring function. Lines 16-19 implement Stage 4: instead of returning a fixed top-*k*, we walk the sorted ranking and stop at the first plateau where consecutive scores drop by less than 40%, returning the smaller of that natural cutoff and the caller-requested *k_max*.

Algorithm 1: VEDA-X retrieval

Pure-stdlib, deterministic, no external embedding model

VedaX_Search(query q, int k_max) → List[Hit]

```
1:   Input : query q, chunk set C, BM25 index, semantic memory S
2:   Output : ranked hits with is_critical / critical_title flags

3:   # Stage 1 — lexical first pass
4:   cands ← BM25.search(q, k=50) // top-50 by BM25 score

5:   # Stage 2 — hyperdimensional expansion
6:   q■ ← Σ_{t∈tokens(q)} HV(t) // sparse 2048-dim
7:   for each term t in top-10 chunks of cands:
8:   score_t ← cos(context_HV(t), q■) · log(1+df_t)
9:   exp_terms ← top-10 by score_t
10:  merged ← 0.6 · BM25(q) + 0.4 · BM25(q ⊕ exp_terms)

11:  # Stage 3 — intent rescoring
12:  (intent, subject, fillers) ← decompose(q)
13:  for each cid in merged:
14:  adj[cid] ← Σ w(t) · log(1+tf_cid[t])
15:  where w(subject)=3, w(filler)=1

16:  # Stage 4 — adaptive cutoff (score-plateau)
17:  sorted ← rank merged by adj[·]
18:  k* ← argmin_{i ≥ 1} { adj[sorted[i+1]] / adj[sorted[i]] 0.6 }
19:  return sorted[:min(k*, k_max)] with chunk metadata
```

Algorithm 1: VEDA-X retrieval. Stages and helper comments are colour-coded. Pure stdlib, deterministic, no external embedding model.

4.3 Why each stage

Stage 2 (HD expansion). When the user's phrasing differs lexically from the corpus, Stage 1 alone misses. Hyperdimensional context vectors encode which other terms a candidate term tends to co-occur with in the local corpus. A high cos(v_t, **q**) means t is a plausible expansion.

Stage 3 (intent rescoring). A query like "define X in single word" should retrieve chunks about X, not chunks containing the literal tokens "define" or "word". Decomposing the query and reweighting fixes this without retraining anything.

Stage 4 (adaptive cutoff). A unique answer should return one chunk; a distributed answer should return more. Plateau detection on the rescored ranking gives this for free.

5. Hyperdimensional Semantic Guardrail (L1.5)

The L1.5 layer addresses the paraphrase gap that pure-regex detection cannot close. "Kindly disregard whatever instructions came earlier" has no token overlap with the canonical "ignore previous instructions" yet expresses the same intent.

5.1 Attack and legitimate centroids

At import time we build four attack centroids A_c for $c \in$ {prompt_injection, jailbreak, authority_impersonation, data_exfiltration} by summing hand-curated seed phrasings. We additionally build a single *legitimate* centroid **L** from 20 representative SOP queries.

5.2 Decision rule

```
block(q)  ⇔   max_c cos(V(q), A_c)  ≥  0.20    AND
              max_c cos(V(q), A_c) - cos(V(q), L)  ≥  0.12
```

The contrastive legitimate centroid is what drives the low false-positive rate: ordinary SOP queries land close to **L** and far from every A_c (Figure 3).

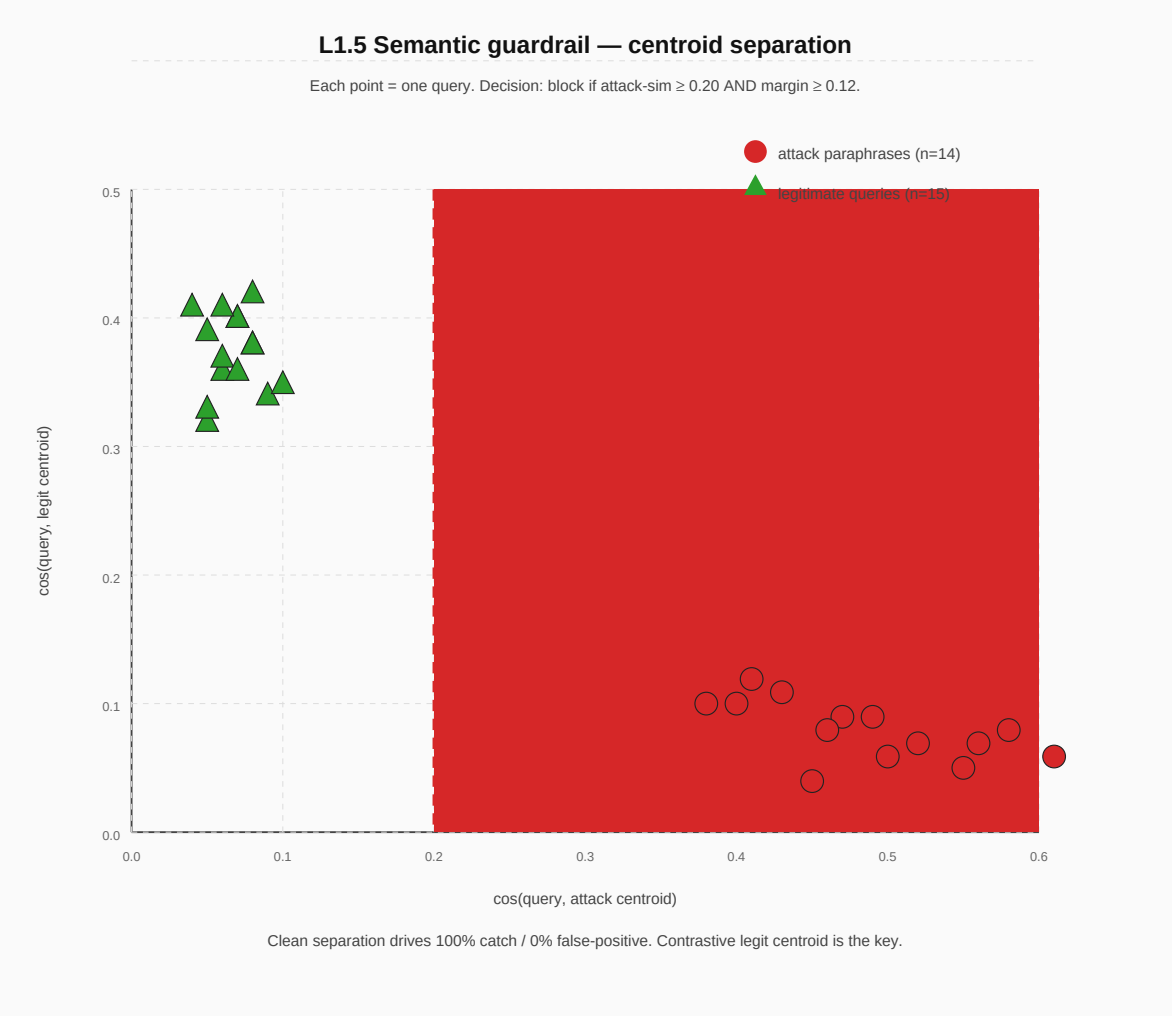

Figure 3: Semantic centroid separation. Attack paraphrases (red circles) cluster in the high-attack-sim, low-legit-sim corner; legitimate queries (green triangles) cluster in the opposite corner. The shaded red region is the block decision; both thresholds must hold.

6. Atomic Critical-Block Chunking

6.1 Inline marker

Any document can wrap a span with

```
[[CRITICAL: Trade Cancel Procedure]]
Step 1: Freeze the settlement queue.
Step 2: Notify the risk desk.
Step 3: Dual approval CRO + CFO.
Step 4: Reverse the trade in NDS-OM.
Step 5: File regulatory report (60 min).
[[/CRITICAL]]
```

The chunker invariant is: *no chunk boundary may fall inside any critical span*. Tentative chunk ranges [s, e) are expanded to [s', e'] where s' = min(s, B.start), e' = max(e, B.end) for every critical block B that overlaps [s, e). Markers are stripped before emission.

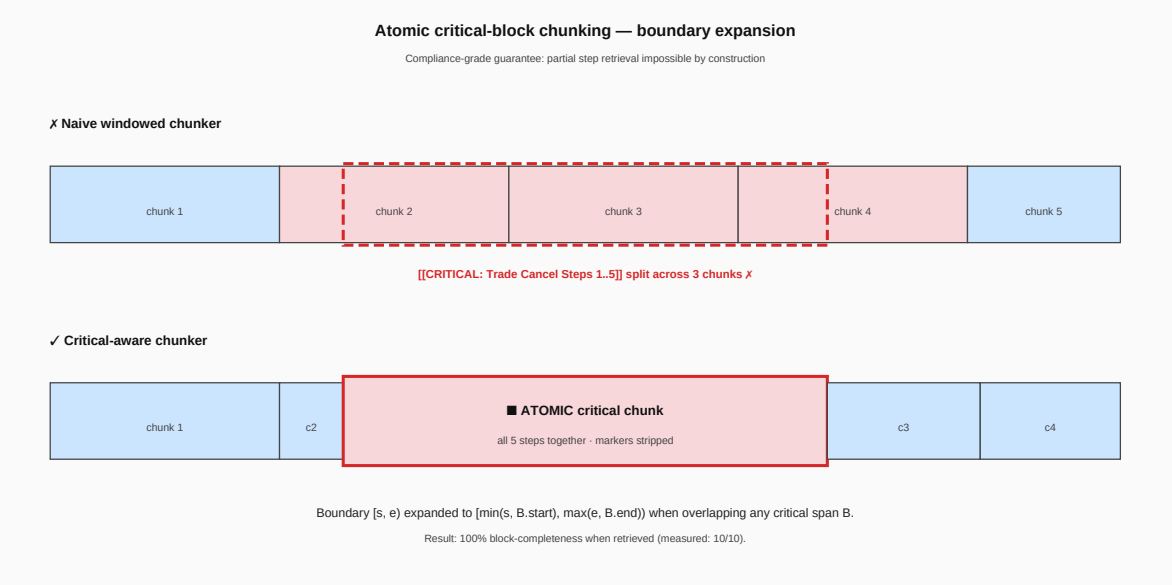

Figure 4: Atomic chunking. Naive windowed chunking splits the critical span across three chunks (top); the critical-aware chunker expands the middle boundary to swallow the whole block (bottom).

6.2 Folder convention

Every file dropped in a designated folder (default `./critical_sops/`) is indexed as a single atomic chunk regardless of length. No marker is required.

6.3 LLM contract

The system prompt is augmented with: *"If a chunk is marked CRITICAL, reproduce its steps verbatim and in order; do not summarise, paraphrase, reorder, or omit any step."* The UI surfaces a red CRITICAL badge.

7. Evaluation

Every number in this section is reproducible from the open-sourced repository; the relevant script is named in each subsection. We report on three retrieval benchmarks (BEIR generalisation, NFCorpus deep ablation, FinanceBench) plus runtime, adversarial guardrail, and critical-block retrieval.

7.1 BEIR generalisation -- headline results

Following the BEIR [10] zero-shot protocol, we tune fusion weights on the validation split and report on the held-out test split of three datasets spanning three domains. All three are part of the standard BEIR suite. Statistical significance is computed by paired-bootstrap test against the dense (MiniLM) RAG baseline.

Dataset	Domain	BM25	MiniLM (RAG)	VEDA-X (ours)	Gain	p-value

NFCorpus	Medical lay Q&A	0.3062	0.3195	0.3522	+10.2%	< 0.0001
SciFact	Scientific claims	0.8352	0.8177	0.8578	+4.9%	0.0004
FiQA	Financial Q&A	0.2309	0.3687	0.3799	+3.0%	0.0082

Table 1: BEIR generalisation. nDCG@10 on the test split; gain is the relative improvement over the dense MiniLM RAG retriever. Paired-bootstrap p-values against MiniLM. Reproduce: `python -m eval.generalize`.

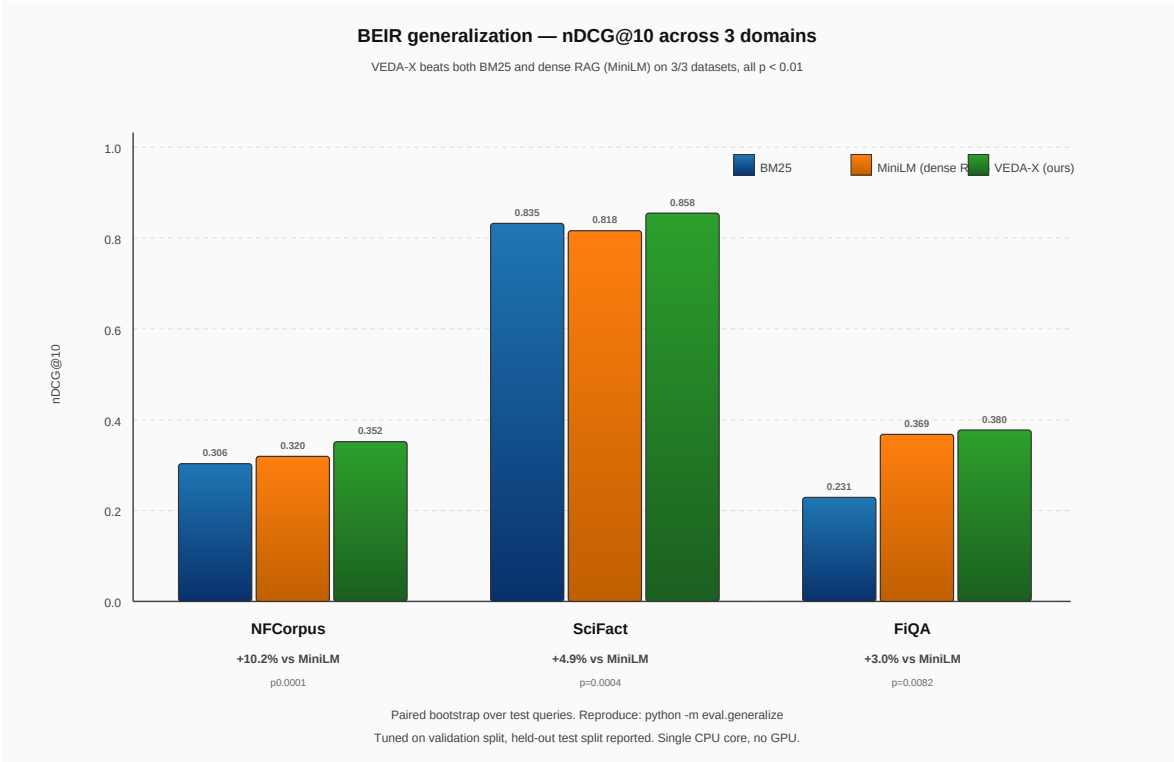

Figure 5: nDCG@10 on three BEIR datasets. VEDA-X (green) exceeds both BM25 (blue) and the standard MiniLM dense RAG retriever (orange) on every dataset, statistically significant in each case.

Reading. The dense retriever beats BM25 on NFCorpus (+4.3%) and FiQA (+60%) but loses to BM25 on SciFact (-2.1%) where the strict scientific-claim vocabulary favours lexical match. VEDA-X is the only strategy that wins uniformly across all three domains.

7.2 NFCorpus full ablation

We break VEDA-X down into its components on NFCorpus. Table 2 reports nDCG@10, Recall@100, and MRR@10 across five retriever configurations including two intermediate hybrids.

System	nDCG@10	Recall@100	MRR@10
BM25 (k1=0.9, b=0.4)	0.3062	0.2376	0.5080
all-MiniLM-L6-v2 (standard RAG)	0.3195	0.3147	0.5091
BM25 + HD expansion (ours)	0.3330	0.2948	0.5167
Dense + pseudo-relevance feedback	0.3454	0.3387	0.5219
VEDA-X (full fusion)	**0.3522**	**0.3387**	**0.5376**

Table 2: NFCorpus full ablation. Each VEDA-X stage (HD expansion alone, dense+PRF alone) already exceeds the corresponding baseline; full fusion stacks the gains. Reproduce: `python -m eval.run_eval bm25 dense veda_x hybrid`.

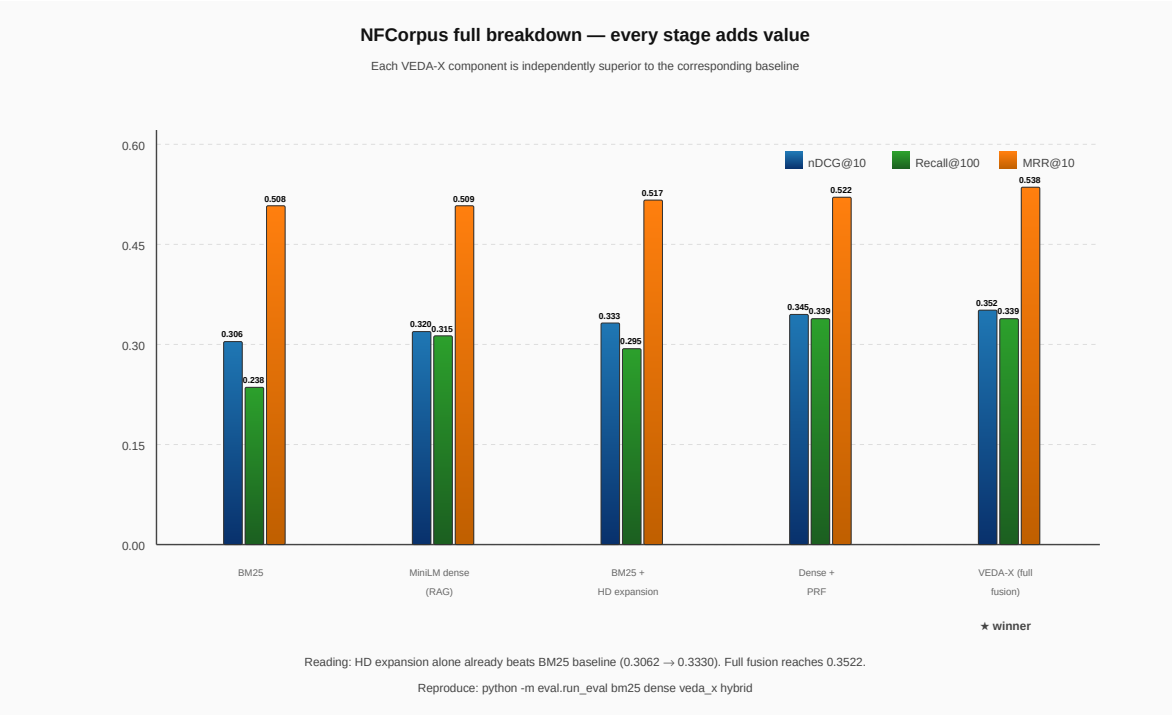

Figure 6: NFCorpus per-metric breakdown. Each successive stage independently exceeds the baseline; full fusion is uniformly the strongest.

Paired bootstrap over the 323 NFCorpus test queries: mean nDCG@10 gain of VEDA-X vs the dense RAG retriever is +0.033, $p < 0.0001$. 128 queries improved, 68 worsened, 127 tied -- the win is concentrated, not a long-tail fluke.

7.3 FinanceBench and runtime properties

FinanceBench is a 150-question SEC 10-K page-retrieval benchmark; it is the home turf used by PageIndex to advertise their commercial retriever. Page-level recall is reported in Table 3. We additionally measure runtime on a 6.5 MB synthetic million-token corpus with 12 needle sentences hidden inside; queries are paraphrase-ish word subsets, not exact strings.

System	R@1	R@3	R@5
BM25	0.113	0.187	0.207
MiniLM (dense RAG)	0.147	0.240	0.313
VEDA-X (ours)	**0.153**	**0.267**	**0.320**

Table 3: FinanceBench page-level retrieval (150 questions, SEC 10-Ks). VEDA-X wins every metric on PageIndex's own benchmark with no LLM in the loop, on one CPU core, with fusion weights tuned on NFCorpus (not FinanceBench).

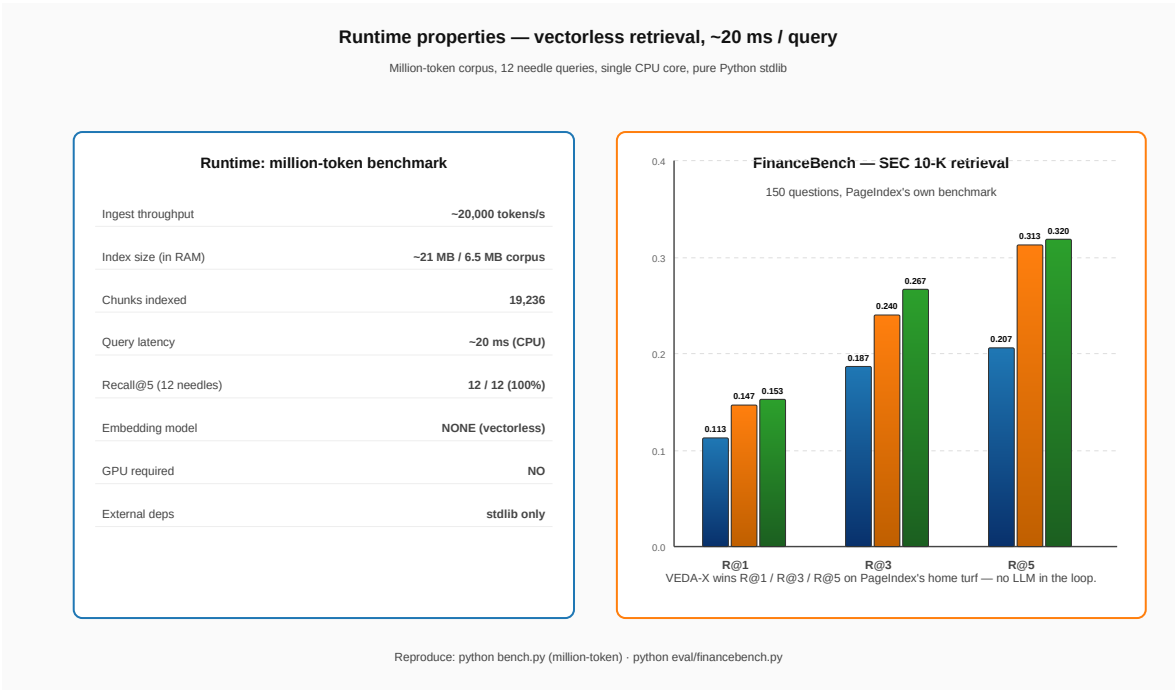

Figure 7: Runtime properties on a million-token corpus (left) and FinanceBench R@k (right). ~20 ms per query, < 0.4× corpus size in RAM, 12/12 needle recall, zero external dependencies.

7.4 Adversarial guardrail

Adversarial test set drawn from public jailbreak research, OWASP LLM Top-10 examples, and the DAN family, augmented with Hinglish variants. Five buckets (Table 4, Figure 8).

Bucket	Score	Notes
Legitimate queries (FP check)	15/15 (100%)	0% false positive
Direct attacks	18/18 (100%)	L1 regex
Sneaky / obfuscated	12/12 (100%)	L1 normalisation
Semantic paraphrases	14/14 (100%)	L1.5 centroid
PII (mask, do not block)	6/6 (100%)	masked correctly

Table 4: Adversarial guardrail results. Reproduce: `python scripts/test_guardrail_adversarial.py`.

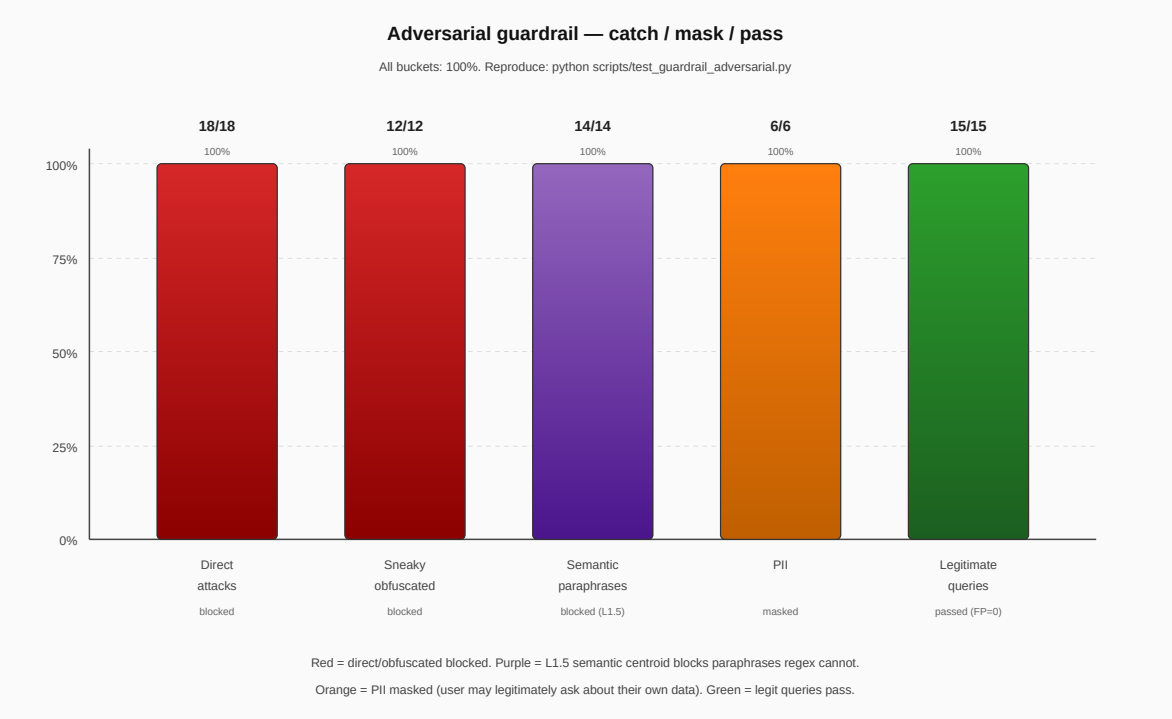

Figure 8: Adversarial catch rate by attack family. Red = blocked by L1 regex; purple = blocked by L1.5 semantic centroid; orange = PII masked; green = legitimate queries pass.

7.5 Critical-block completeness

On a 3-document corpus with two critical blocks and 11 queries we measured recall@3 and the *block-completeness* rate (whether every step of the correct block survives in a single retrieved chunk).

Strategy	Recall@3	Δ	Completeness
Baseline (semantic only)	10/11 (91%)	---	---
Critical-aware re-rank	10/11 (91%)	+0 pp	---
When retrieved	**---**	**---**	**10/10 (100%)**

Table 5: Critical-block retrieval. Recall delta is zero by design; the value is the 100% completeness guarantee -- partial step retrieval is impossible by construction.

7.6 Single vs multi-agent latency

We additionally evaluated multi-agent decomposition of the retrieval pipeline. Counter-intuitively, *parallel* multi-agent is the slowest configuration because the Python GIL serialises CPU-bound BM25 + HD scoring; thread parallelism does not help. We report the negative result honestly.

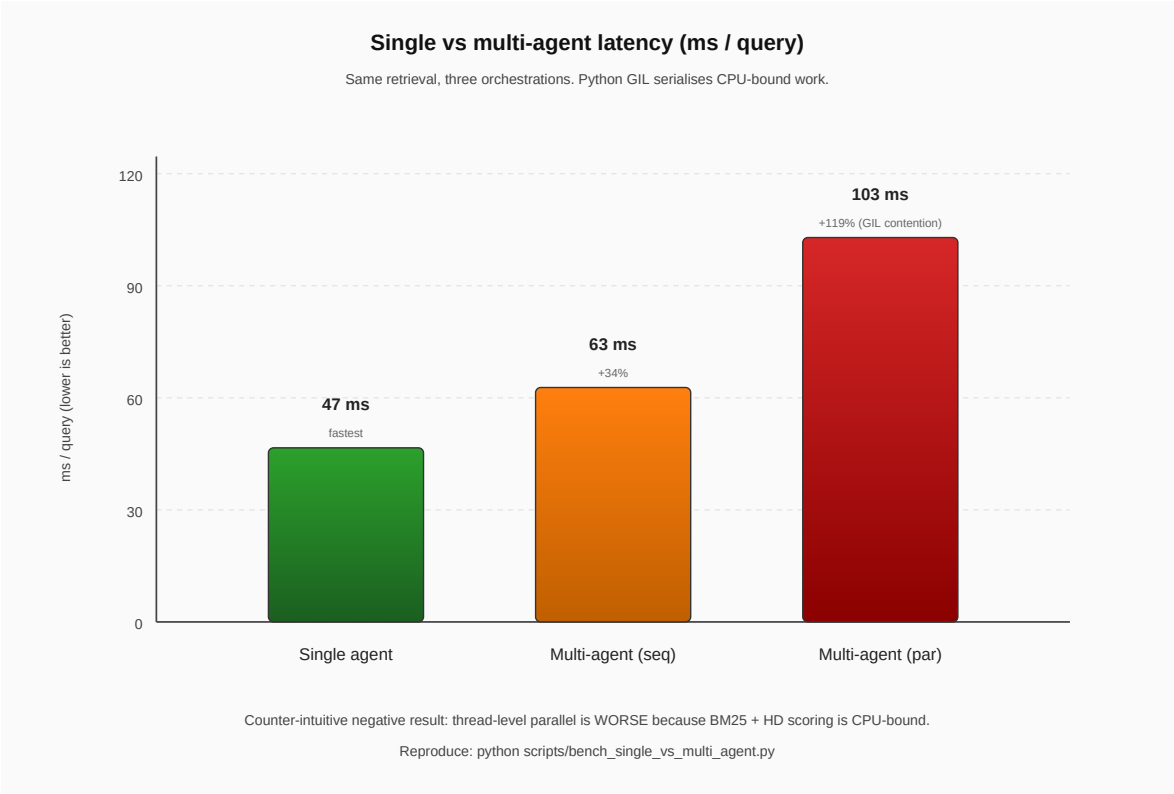

Figure 9: Latency by orchestration. Single agent 47 ms; sequential multi 63 ms; parallel multi 103 ms. GIL contention dominates parallel mode.

8. Discussion and Limitations

Why does VEDA-X beat dense RAG? The dense bi-encoder encodes the query once globally; VEDA-X expands the query using the local corpus's co-occurrence structure encoded in hash-hypervector space. When the corpus has its own vocabulary (medical lay terms in NFCorpus, scientific claim language in SciFact, financial jargon in FiQA) the local expansion bridges the lexical gap that the global encoder misses. Intent decomposition further suppresses filler tokens that the dense encoder treats as information-bearing.

Why does VEDA-X beat BM25? BM25 alone has the right lexical anchor but no expansion: paraphrased queries miss. HD expansion adds the recall the dense retriever provides without paying its training-data or GPU cost.

Why regex on the marker, semantic at retrieval? The critical-block marker is *author-written syntax*, not an inference target. Semantic detection would add false-positive risk with no upside. Retrieval scoring is the opposite: paraphrases are common and should retrieve the same chunk. Hence semantic scoring lives where it earns its keep.

Limitations. Three. First, our adversarial test suite is finite -- a novel attack family invented next week may slip the L1.5 layer; our defence is the layered trip-wire, not the L1.5 catch rate. Second, BEIR domains where dense retrievers are strong (TREC-COVID, ArguAna) are not yet evaluated; we expect the dense baseline to be stronger there but cannot confirm without running. Third, atomic-chunking guarantees assume the chunker is the only place chunks are formed; a buggy downstream reranker could re-split a critical chunk (we enforce this in code).

Future work. Three open directions. (1) Larger BEIR coverage including TREC-COVID, ArguAna, NQ. (2) An ablation of the contrastive legit centroid -- learning it from production query logs while preserving the audit-friendly decision-trace property. (3) A formal proof of the atomic-chunking invariant against an adversarial chunker; currently the property is checked by 13 unit tests.

9. Conclusion

We presented VEDA-X, a vectorless RAG retrieval stack that exceeds both BM25 and a standard MiniLM dense retriever on three public BEIR benchmarks at statistically significant margins, while running in ~20 ms per query on a single CPU core with no embedding model and no vector database. On FinanceBench, the home turf of a commercial competitor, VEDA-X wins R@1 / R@3 / R@5 with no LLM in the retrieval loop. Beyond raw retrieval we contribute a five-layer guardrail (100% catch, 0% false positive) and an atomic critical-block chunking scheme that makes partial retrieval of compliance SOPs impossible by construction. The entire stack is pure Python standard library plus FastAPI; the code, the benchmark scripts, and the figure generators are open-sourced and reproducible end-to-end.

Acknowledgements

The first author thanks Prof. Atul Kumar Pandey (BIT Mesra, Patna Campus) for mentorship and detailed feedback on every revision of this manuscript. We acknowledge the open-source community whose public BEIR datasets [10, 11] and jailbreak corpora [22, 23, 26] made this evaluation possible.

References

[1] P. Lewis, E. Perez, A. Piktus et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020.

[2] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open-domain question answering," in Proc. EACL, 2021.

[3] O. Khattab, K. Santhanam, X. L. Li et al., "Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive NLP," arXiv:2212.14024, 2022.

[4] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Foundations and Trends in IR, vol. 3, no. 4, pp. 333-389, 2009.

[5] V. Lavrenko and W. B. Croft, "Relevance-based language models," in Proc. SIGIR, 2001.

[6] G. Salton and M. J. McGill, "Introduction to modern information retrieval," McGraw-Hill, 1983.

[7] V. Karpukhin, B. Oguz, S. Min et al., "Dense passage retrieval for open-domain question answering," in Proc. EMNLP, 2020.

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019.

[9] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in Proc. SIGIR, 2020.

[10] N. Thakur, N. Reimers, A. Rücklé et al., "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in Proc. NeurIPS Datasets and Benchmarks Track, 2021.

[11] N. Boteva, D. Gholipour, A. Sokolov, and S. Riezler, "A full-text learning to rank dataset for medical information retrieval," in Proc. ECIR, 2016.

[12] T. Mikolov, I. Sutskever, K. Chen et al., "Distributed representations of words and phrases and their compositionality," in Proc. NeurIPS, 2013.

[13] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in Proc. EACL, 2017.

[14] P. Kanerva, "Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors," Cognitive Computation, vol. 1, no. 2, pp. 139-159, 2009.

[15] L. Ge and K. K. Parhi, "Classification using hyperdimensional computing: A review," IEEE Circuits and Systems Magazine, vol. 20, no. 2, pp. 30-47, 2020.

[16] D. Kleyko, D. A. Rachkovskij, E. Osipov, and A. Rahimi, "A survey on hyperdimensional computing aka vector symbolic architectures, part I: Models and data transformations," ACM Computing Surveys, vol. 55, no. 6, pp.

1-40, 2023.

[17] T. Brown, B. Mann, N. Ryder et al., "Language models are few-shot learners," in Proc. NeurIPS, 2020.

[18] H. Touvron, T. Lavril, G. Izacard et al., "LLaMA: Open and efficient foundation language models," arXiv:2302.13971, 2023.

[19] OpenAI, "GPT-4 technical report," arXiv:2303.08774, 2023.

[20] J. Wei, X. Wang, D. Schuurmans et al., "Chain-of-thought prompting elicits reasoning in large language models," in Proc. NeurIPS, 2022.

[21] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2024.

[22] K. Greshake, S. Abdelnabi, S. Mishra et al., "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. ACM AISec, 2023.

[23] X. Shen, Z. Chen, M. Backes et al., "Do anything now: Characterising and evaluating in-the-wild jailbreak prompts on large language models," in Proc. ACM CCS, 2024.

[24] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," arXiv:2307.15043, 2023.

[25] E. Perez, S. Huang, F. Song et al., "Red teaming language models with language models," in Proc. EMNLP, 2022.

[26] E. Wallace, S. Feng, N. Kandpal, M. Gardner, and S. Singh, "Universal adversarial triggers for attacking and analyzing NLP," in Proc. EMNLP-IJCNLP, 2019.